

Reviewer Pack — Cheeger Constant Inverse Proportional to Extended Frege Proo...

Entry bd4677253c0f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 07:16:00 UTC

Cheeger Constant Inverse Proportional to Extended Frege Proof Size

Entry ID: bd4677253c0f Verdict: INCONCLUSIVE Recorded: 2026-05-11 07:16:00 UTC

1. Conjecture

Field A (mathematical branch): Spectral Geometry **Field B** (complexity object): Extended Frege Proof Size

Statement:

For a 3-CNF instance Φ with n variables, the Cheeger constant $h(\Phi)$ of its hypergraph representation satisfies $h(\Phi) = \Theta(1 / \text{size}(\Pi))$, where Π is the minimal Extended Frege proof of Φ .

Rationale (proposer’s reasoning):

The Cheeger constant, a spectral geometry invariant, captures the expansion properties of the hypergraph. In proof complexity, expansion properties might correlate with the required proof size, as highly connected structures may necessitate longer proofs.

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 660357a86dd45107

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]

        # Eliminate below pivot
        for j in range(i+1, n):
            factor = -A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] += factor * A[i][k]
            b[j] += factor * b[i]

    # Back-substitute
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
    return x

def cheeger_constant(adj_list):
    n = len(adj_list)
    degree = [sum(1 for _ in neighbors) for neighbors in adj_list]
    laplacian = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            if j in adj_list[i]:
                laplacian[i][j] = laplacian[j][i] = -1
                degree[i] -= 1
                degree[j] -= 1
    laplacian = [[d - A[i][j] for j, d in enumerate(degree)] + [0] * (n - len(adj_list[i])) for i,
```

```

eigenvalues = sorted(gaussian_elimination(laplacian, [0] * n))
return eigenvalues[1]

def extended_frege_proof_size(phi):
    # Placeholder function to estimate proof size
    # This is a dummy implementation and should be replaced with actual DPLL-based algorithm
    return random.randint(10, 100)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.choice([5, 10, 15, 20, 30, 40])
    phi = [random.choice([True, False]) for _ in range(n)]
    adj_list = [[] for _ in range(n)]

    # Generate a random 3-CNF instance
    for i in range(n):
        if phi[i]:
            continue
        for j in range(i+1, n):
            if phi[j]:
                continue
            for k in range(j+1, n):
                if phi[k]:
                    continue
                adj_list[i].append(j)
                adj_list[i].append(k)
                adj_list[j].append(i)
                adj_list[j].append(k)
                adj_list[k].append(i)
                adj_list[k].append(j)

    h_phi = cheeger_constant(adj_list)
    size_pi = extended_frege_proof_size(phi)

    return {
        "metric_name": "Cheeger Constant Inverse Proportional to Proof Size",
        "metric_value": h_phi * size_pi,
        "instances_tested": 1,
        "conjecture_holds": h_phi * size_pi <= 1,
        "counterexample": "" if h_phi * size_pi <= 1 else f"Counterexample:  $h(\Phi)=\{h\_phi\}$ ,  $size(\Pi)=$ 
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value)**2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

```

```

if all(res["conjecture_holds"] for res in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not res["conjecture_holds"] for res in results) and support_fraction >= 0.8:
    first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5952b1b6.py", line 103, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5952b1b6.py", line 86, in run_trial
    h_phi = cheeger_constant(adj_list)
              ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5952b1b6.py", line 52, in cheeger_constant
    laplacian = [[d - A[i][j] for j, d in enumerate(degree)] + [0] * (n - len(adj_list[i])) for i, adj_list in enumerate(adj_list)]
                  ^
NameError: name 'A' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to undefined variable 'A' in cheeger_constant calculation, preventing data collection | next: Fix the NameError in the cheeger_constant function by properly defining adjacency matrix 'A' before computing Laplacian

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	96707
2	preregistration	ollama_remote	qwen3:8b	0	0	24264
3	novelty	ollama_remote	qwen3:8b	0	0	20932
4	novelty	ollama_remote	qwen3:8b	0	0	15190

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21033
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12458
7	judge	ollama_remote	qwen3:8b	0	0	16869

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 207452 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/bd4677253c0f.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/bd4677253c0f.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/bd4677253c0f.tar.gz (if generated)